Generative machine learning approaches to optimization

Victor Alves and John R. Kitchin*

Department of Chemical Engineering, Carnegie Mellon University, Pittsburgh, PA 15213

E-mail: jkitchin@andrew.cmu.edu

Abstract

Solving optimization problems, especially for nonlinear and constrained systems, is a challenge. Decades of specialized algorithms have been developed for general and special cases of root finding, minimization (including constraints), for parameter estimation, and mapping connected spaces. These approaches typically require a model, or set of equations, and then analytical, or iterative numerical approaches can be used to find a solution, and in some special cases a globally best solution can be found and proven. In this work we present an alternative approach to solving these problems that is based in generative machine learning models. The idea is these models either estimate a joint probability distribution of input and output variables, or are able to transform a simple distribution to a target distribution. Then, by suitable conditioning (specifying desired properties of some variables), the solutions to these problems can be obtained by sampling the distribution, or by transformation of a distribution sample to the solution space. We illustrate the approach with Gaussian mixture models, which approximate the joint probability distribution and conditional flow matching which uses the distribution transformation approach. We show examples in root finding, unconstrained and constrained optimization, parameter estimation and mapping input and output spaces. This work is intended to be pedagogical to show how generative

models can be used to solve problems in optimization. We discuss some limitations of this approach, but conclude the approach has promise and is different than existing approaches.

Introduction

The field of optimization is immense, with elegant solutions tailored to specific problem applications, depending on the nature of the objective function and constraints that are to be solved (e.g., linear *vs.* nonlinear, convex *vs.* nonconvex, differentiable *vs.* nondifferentiable), the nature of the variables involved in the optimization problem (discrete, continuous, or both), and lastly, the nature of the solution itself, global *vs.* local optimization. This myriad of problems that arise in many fields of science and engineering gave birth to several solution techniques, ranging from the seminal work of linear programming and the Simplex method by Dantzig,[1] to the advent of robust nonlinear programming techniques including sequential quadratic programming (SQP),[2–4] and the popularization of interior-point methods.[5] Additionally, solutions of mixed integer nonlinear programming (MINLP) algorithms[6] and recent innovations on generalized disjunctive programming (GDP) which embeds logical decisions into mathematical programming formulations,[7] and global optimization[8–10] are all examples of how rich is the literature on the topic.In Python the `scipy` library is particularly notable,[11] but others also exist like `Pyomo`,[12] GEKKO,[13] Gurobi,[14] Mosek,[15] and JuMP.[16] These libraries include both algebraic optimization modeling languages as well as commercial and/or open-source optimization solvers. Despite all the work aforementioned, the list is by no means exhaustive, and many other interesting research directions are currently available and being actively developed.

We see a particular interest in algorithmic development of solutions to optimization problems that are specific to the nature of the problem, its variables and local/optimal convergence criteria. However, less attention has been given to the use of emerging and powerful tools such as generative machine learning (genML) in the well-developed area of optimiza-

tion. It is clear that optimization/mathematical programming is one of the cornerstones of machine learning (independent of the generative nature or lack thereof), but the reverse application, that is, genML as a tool to solve optimization problems, seems to be largely unexplored.

To orient the reader on where a generative-based optimization approach fits in the optimization landscape, we illustrate genML-opt in Figure 1, based on the classification of optimization problems in Ref. 17. We intentionally modified the original flowchart in Refs. 17,18 to show which class of problems we are able to currently employ genML-opt. Although promising, there is still a lot to cover when compared to the current state-of-the-art in optimization.

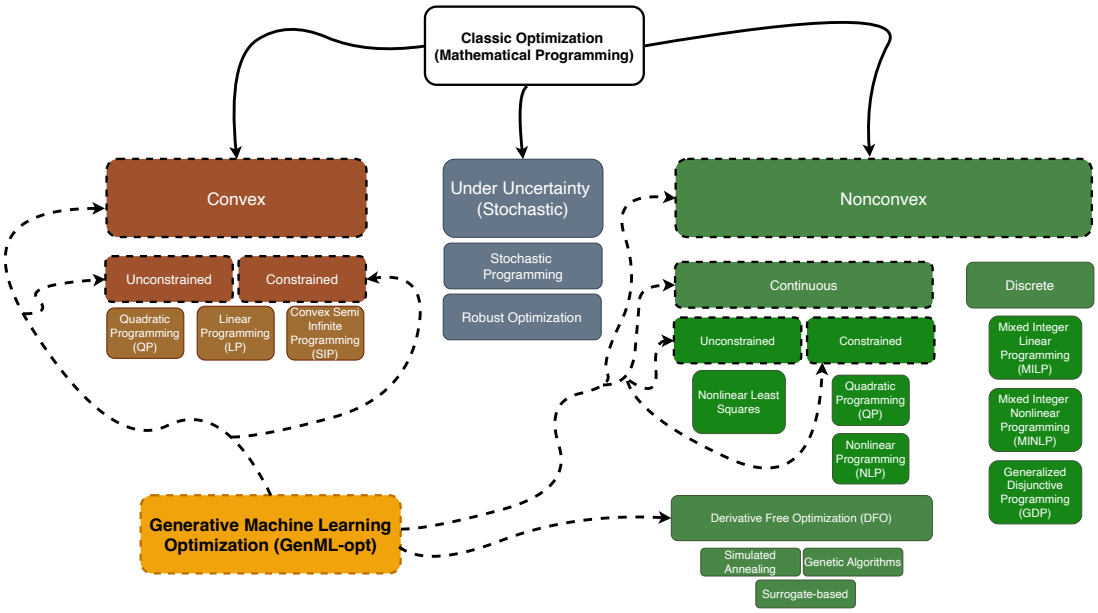


Figure 1: Tree containing classes of optimization problems, based on flowcharts available in the literature.[17,18] We show with connected dashed lines and also highlighting classes of optimization problems that we were able to currently solve using generative machine learning applied to optimization - GenML-opt.

In this manuscript we explore a different way to think about optimization through generative machine learning (genML).[19] The basic concept in genML is that if one knows the joint probability distribution of a set of variables, then it is possible to generate samples of that distribution. Alternatively, if one knows how to transform a reference distribution to the

joint probability distribution of those variables, then one samples the reference distribution and transforms it to a sample of the desired distribution.

In fact, generative models have been applied to optimization problems before. Invertible neural networks are a special class of models that are used to model the joint distribution of inputs and outputs.[20] Once they are trained, feasible solutions can be found by sampling them. Diffusion models have been applied to inverse problems in mechanics.[21] We recently applied genML to solve an inverse linear problem.[22] Generative models can also be used to enhance combinatorial optimization problems[23] where they can be used to generate near-optimal solutions that are subsequently refined by conventional algorithms. Finally, generative adverserial models have been used in optimization.[24]

Probably the most common genML method used today is in the generation of text[25] and images.[26] Large language models (LLMs) are a form of genML where the most likely sequence of words to follow a prompt is generated. LLMs are not the focus of this work. In image generation, a genML model is trained on a large set of images to "learn the image distribution", and then images are generated by sampling that distribution. We find it helpful to think of an image first as a matrix of pixel values (e.g. the RGB intensities), and then to think of that matrix as a flattened vector. Then, a large set of images will have a distribution of values in that vector space where there are correlations (covariance) between the elements of the vector that define an image space. Thus, one generates a sample vector, and then reshapes it into a matrix, which can be visualized as an image. This is the motivation for the present work.

That does not sound immediately helpful in solving optimization problems until we note two things. First, instead of a collection of pixel values, the vector elements may be inputs and outputs of a model or system, and by concatenating those we obtain a vector analogous to the flattened image vector. The vector elements may even include derivatives of the model, or other easily derived features. If we have a large number of these vectors, then they will form a distribution in that vector space where there are correlations (covariance)

between the elements, e.g. the inputs and outputs. Since it is evident generative models can learn that distribution for images, it seems reasonable they could learn this distribution for inputs/outputs, which we demonstrate in this work.

Thus, if we can learn or estimate the joint probability distribution, we can generate samples of inputs and outputs from it. Second, we must introduce the idea of conditional generation, that is, we can specify some values of the variables, update the joint probability distribution of the remaining variables, and then generate those variables consistent with our condition. Thus, if we condition on inputs, we can then generate samples of the output, and vice versa. This is directly related to root finding; we might condition on the output being zero, and generate the inputs that lead to that result.

To connect this idea to finding minima/maxima we simply need to see the connection between inputs of a model and its derivatives. If we know that joint probability distribution, then we generate samples conditioned on the derivatives being equal to zero. We can include constraints by leveraging a Lagrangian function, or another augmented function and condition the derivatives of that to be zero. Finally, in parameter estimation, we need the joint probability distribution between the parameters and an observed output(s). Then to estimate parameters from an observation, we simply condition on the observation to find the parameters. These are all related to solving inverse problems in optimization.

An alternative approach to generative modeling is to transform a simple distribution into a more complex distribution. In diffusion and flow-matching models, this transformation is learned from data, and then it is used conditionally to generate samples of a target distribution (e.g. the outputs) from samples of a simpler distribution such as a normal distribution.

We think this approach is fundamentally different than a conventional view point. There may be a model involved, but it is not necessary; the approach can be completely data-driven in some cases. We do not need to think about a forward or inverse model. The joint probability distribution (or the transformation function) is simultaneously both of these;

conditioning determines what direction if any one thinks in (for example, it is possible to condition on some inputs and outputs). That means we get solutions to optimization problems without using conventional approaches like Newton-Raphson iterative methods, or other specialized algorithms like simplex methods to solve nonlinear programs. We make no claims of superiority of generative methods over the well-established methods; indeed, conventional algorithms offer many advantages to the ideas presented here. Instead, we focus on developing a proof of concept for a new way to think about solving these problems. We do this through a series of examples that span root finding, minimization including constraints, parameter estimation, and state mapping.

GenML has the potential to serve as an alternative optimization solution approach to several classes of problems listed above, including linear and nonlinear programming (both constrained and unconstrained), parameter estimation problems, inverse mapping, and root finding, to name a few. Intriguingly, we have observed that the solution approach to these types of problems is remarkably consistent, as opposed to highly-tailored and specific algorithms (the current *status quo*) as long as data is available. We do not anticipate that generative machine learning formulations will replace classic mathematical programming development. Instead, we believe that a branch of optimization problems that can be solved by using GenML will employ even more sophisticated optimization algorithms at the training and inference of generative models. In short, the optimization layer will always be present, and its interaction with the researcher/practitioner will be what might change within the next years.

Methods

Gaussian mixture models

Gaussian Mixture Models (GMMs) are a powerful and versatile mathematical tool that is able to characterize complex and multimodal distributions by combining multiple individ-

ual Gaussians. They are used in machine learning applications related but not limited to data clustering.[27,28] They are also recognized in the literature related to pattern recognition in machine learning.[29] Furthermore, in the field of robotics, Gaussian Mixture Regression (GMR) has been used as a generative modeling framework,[30] and it is considered a generative model.[31] We follow the notation from Ref 30 for the mathematical definition of GMMs. However, the reader should note that equally valid formulations are present in the literature.[29,31]

From a machine learning perspective, the main task is, given a pair of vectors x and y, to learn a probability distribution as defined as in Eq. 1, which is the definition of a Gaussian mixture model.

$$p(\boldsymbol{x}, \boldsymbol{y}) = \sum_{k=1}^{K} \pi_k \mathcal{N}_k \left(\boldsymbol{x}, \boldsymbol{y} \mid \boldsymbol{\mu}_{\boldsymbol{xy}_k}, \boldsymbol{\Sigma}_{\boldsymbol{xy}_k}\right) \tag{1}$$

In which $\mathcal{N}_k \left(\boldsymbol{x}, \boldsymbol{y} \mid \boldsymbol{\mu}_{\boldsymbol{xy}_k}, \boldsymbol{\Sigma}_{\boldsymbol{xy}_k}\right)$ are the k^{th} Gaussian distributions in a mixture, which are defined by their respective means $\boldsymbol{\mu}_{\boldsymbol{xy}_k}$ and covariances $\boldsymbol{\Sigma}_{\boldsymbol{xy}_k}$. $\pi_k \in [0, 1]$ are defined as *weights* or *priors*, which quantify the contribution of each Gaussian to the mixture model.

Training

The training step corresponds to obtaining an accurate prediction of this unknown, probably multimodal and multivariable probability distribution, given a dataset defined by the pair of vectors x, y. For simplicity, we can lump the parameters related to the formulation of Eq. 1 which includes the means, covariances and mixture weights as $\theta := \left\{\pi_k, \boldsymbol{\mu}_{\boldsymbol{xy}_k}, \boldsymbol{\Sigma}_{\boldsymbol{xy}_k} : k = 1, \ldots, K\right\}$.

As a figure of merit is needed to fit this generative machine learning model, we resort to a likelihood formulation, defined as in Eq. 2.

$$p(x, y \mid \boldsymbol{\theta}) = \prod_{n=1}^{N} p\left(x_n, y_n \mid \boldsymbol{\theta}\right), \quad p\left(x_n, y_n \mid \boldsymbol{\theta}\right) = \sum_{k=1}^{K} \pi_k \mathcal{N}\left(x, y \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\right) \tag{2}$$

The log-likelihood[31] is formulated in Eq. 3. Ideally, one could solve for $\mathrm{d}\mathcal{L}/\mathrm{d}\boldsymbol{\theta} = 0$ and

the optimal parameters θ^* that maximizes the likelihood would be obtained analytically. However, since Eq. 3 is not a closed-form expression/solution,[31] an iterative method is used. Expectation Maximization (EM) is applied in GMMs sucessfully and this is what packages such as `gmr`[30] and the scikit-learn[32] implementation of GMM does.

$$\log p(\boldsymbol{x}, \boldsymbol{y} \mid \boldsymbol{\theta}) = \sum_{n=1}^{N} \log p\left(\boldsymbol{x}_n \mid \boldsymbol{\theta}\right) = \underbrace{\sum_{n=1}^{N} \log \sum_{k=1}^{K} \pi_k \mathcal{N}\left(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\right)}_{=:\mathcal{L}} \tag{3}$$

Prediction

Following the conditional distribution property of each individual Gaussian, one is able to compute the conditional distribution $p(y \mid x, \theta)$ by first writing down the conditional distribution for a given k^{th} element.[30]

$$\boldsymbol{\mu}_{\boldsymbol{x}\boldsymbol{y}_k} = \begin{bmatrix} \boldsymbol{\mu}_{\boldsymbol{x}k} \\ \boldsymbol{\mu}_{\boldsymbol{y}k} \end{bmatrix}, \quad \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{y}_k} = \begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{x}_k} & \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{y}_k} \\ \boldsymbol{\Sigma}_{\boldsymbol{y}\boldsymbol{x}_k} & \boldsymbol{\Sigma}_{\boldsymbol{y}\boldsymbol{y}_k} \end{bmatrix} \tag{4}$$

The conditional distribution of $\boldsymbol{y}$ given $\boldsymbol{x}$ for the k^{th} Gaussian component is defined as in Eq.5:

$$\begin{aligned} \boldsymbol{\mu}_{\boldsymbol{y}|\boldsymbol{x}}^{(k)} &= \boldsymbol{\mu}_{\boldsymbol{y}k} + \boldsymbol{\Sigma}_{\boldsymbol{y}\boldsymbol{x}_k} \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{x}_k}^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_{\boldsymbol{x}k}) \\ \boldsymbol{\Sigma}_{\boldsymbol{y}|\boldsymbol{x}}^{(k)} &= \boldsymbol{\Sigma}_{\boldsymbol{y}\boldsymbol{y}_k} - \boldsymbol{\Sigma}_{\boldsymbol{y}\boldsymbol{x}_k} \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{x}_k}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{y}_k} \end{aligned} \tag{5}$$

To obtain the full conditional distribution $p(\boldsymbol{y} \mid \boldsymbol{x})$, we combine the component-wise conditional Gaussians using the posterior weights $\pi_{k|\boldsymbol{x}}$, which represent the probability of component k given the observed $\boldsymbol{x}$:

$$\pi_{k|\boldsymbol{x}} = \frac{\pi_k \, \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_{\boldsymbol{x}k}, \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{x}_k})}{\sum_{l=1}^{K} \pi_l \, \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_{\boldsymbol{x}l}, \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{x}_l})} \tag{6}$$

Finally, the final predicted distribution corresponds to a mixture of conditional Gaussians:

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \sum_{k=1}^{K} \pi_{k|\boldsymbol{x}} \mathcal{N}(\boldsymbol{y} \mid \boldsymbol{\mu}_{\boldsymbol{y}|\boldsymbol{x}}^{(k)}, \boldsymbol{\Sigma}_{\boldsymbol{y}|\boldsymbol{x}}^{(k)}) \tag{7}$$

If one desires to have them in a more compact form, we refer to our previous definition of the lumped hyperparameters as θ as

$$\theta =: \left\{\pi_k, \boldsymbol{\mu}_{\boldsymbol{x}\boldsymbol{y}_k}, \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{y}_k}\right\}_{k=1}^{K} \tag{8}$$

which yields a predictive distribution form as in Eq. 9.

$$p(\boldsymbol{y} \mid \boldsymbol{x}; \theta) = \sum_{k=1}^{K} \pi_{k|\boldsymbol{x}}(\theta) \mathcal{N}(\boldsymbol{y} \mid \boldsymbol{\mu}_{\boldsymbol{y}|\boldsymbol{x}}^{(k)}, \boldsymbol{\Sigma}_{\boldsymbol{y}|\boldsymbol{x}}^{(k)}) \tag{9}$$

Eq. 9 plays a major role in our work. As we draw samples based on conditioning, we can simultaneously infer in variables that belong to both the x or y spaces. We show this in several applications related to optimization.

Lastly, we note that typical optimization techniques are used, more specifically during the training step of the generative approach, namely the EM algorithm.[33] However, this optimization step that requires $\mathcal{O}(\mathcal{D}^3)$ for a dataset of dimensionality $\mathcal{D}$, is performed offline and separately from the applications in which the GM model is employed.

Conditional normalizing flow matching

Flow matching is a simulation-free approach to training continuous normalizing flows.[34] Unlike traditional normalizing flows that require computing expensive Jacobian determinants, flow matching learns a time-dependent vector field $v_t(x)$ that transforms samples from a simple base distribution p_0 (typically $\mathcal{N}(0, I)$) to a target data distribution p_1 through ordinary differential equations (ODEs).

Given a data sample $x_1 \sim p_1(x)$ and a base sample $x_0 \sim p_0(x)$, we construct a probability path $p_t(x)$ for $t \in [0, 1]$ that interpolates between the two distributions. A common choice

is the conditional Gaussian path:

$$p_t(x|x_1) = \mathcal{N}(x|tx_1, (1-(1-\sigma)t)^2 I) \tag{10}$$

where σ is a small constant (typically 10^{-5}) to ensure numerical stability.

The corresponding conditional vector field that generates this path is:

$$u_t(x|x_1) = \frac{x_1 - (1-\sigma)x}{1-(1-\sigma)t} \tag{11}$$

For optimization problems, we extend flow matching to learn conditional distributions $p(x|c)$, where x represents the decision variables and c represents the conditioning information (e.g., constraint values, target objectives, or system parameters).[35] This allows us to sample from different conditional distributions by simply changing the conditioning value.

The conditional flow matching objective is:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,p_1(x_1,c),p_0(x_0)}\left[\|v_\theta(x_t,c,t) - u_t(x_t|x_1)\|^2\right] \tag{12}$$

where $v_\theta(x_t, c, t)$ is the learned vector field parameterized by a neural network with parameters θ, $t \sim \text{Uniform}(0,1)$ is the time variable, $x_t \sim p_t(x|x_1)$ is a sample along the flow path, and (x_1, c) are paired samples from the training data.

We parameterize the vector field $v_\theta(x, c, t)$ using a multi-layer neural network with the following architecture: the Input (concatenation of $[x, c, t]$), the hidden layers (3 fully-connected layers with 64 units each and sigmoid activation functions), and the output (a vector field prediction $v \in \mathbb{R}^{d_x}$).

The choice of sigmoid activations (rather than ReLU) provides smooth gradients throughout the network, which is beneficial for learning continuous vector fields.[36] The model is trained using the following procedure. First, for each training sample, we store the input-output pair (x, c) where x represents the solution and c represents the conditioning informa-

tion. During training, each mini-batch involves sampling data pairs (x_1, c) from the training set, sampling base noise $x_0 \sim \mathcal{N}(0, I)$, and sampling time $t \sim \text{Uniform}(0, 1)$. We then construct the intermediate state $x_t = tx_1 + (1 - (1 - \sigma)t)x_0$ and compute the target vector field $u_t(x_t|x_1) = \frac{x_1 - (1-\sigma)x_t}{1-(1-\sigma)t}$. The training objective minimizes the mean squared error between the predicted and target vector fields using the Adam optimizer[37] with learning rate 10^{-3}. To improve training stability, all input features are standardized to have zero mean and unit variance before training.

Training typically requires 500 epochs with batch size 64 for the problems considered in this work. To generate samples from the learned conditional distribution $p(x|c)$, we solve the ODE:

$$\frac{dx}{dt} = v_\theta(x, c, t), \quad x(0) \sim \mathcal{N}(0, I) \tag{13}$$

from $t = 0$ to $t = 1$ using the Euler method with $N = 100$ steps:

$$x_{t+\Delta t} = x_t + v_\theta(x_t, c, t) \cdot \Delta t, \quad \Delta t = \frac{1}{N} \tag{14}$$

The final state $x(1)$ represents a sample from $p(x|c)$. To obtain multiple samples, we simply repeat this process with different initial noise samples $x(0)$.

Conditional flow matching offers several advantages for optimization problems. First, unlike score-based models[38] or traditional normalizing flows,[39] flow matching provides simulation-free training that does not require computing expensive score functions or Jacobian determinants during training. Second, the learned distribution naturally captures multi-modal solutions, enabling exploration of the entire solution space when constraints admit multiple feasible points. Third, the flexible conditioning mechanism allows rapid sampling of solutions for different problem instances by simply changing the conditioning value c without retraining the model. Fourth, the distribution of samples provides natural uncertainty quantification for the solutions. For problems with missing data or regions of low sampling density, flow matching learns to interpolate smoothly through these regions by leveraging

the continuous nature of the learned vector field.

LLM usage in this work

We used Claude Code with the Opus 4.5 model extensively to develop the code in this work and for literature review. The manuscript was largely written by the authors. The GMM code used in this work was originally written by the authors, and then integrated into the work by Claude Code. The conditional normalizing flow code and the narrative describing it in the manuscript was created with Claude Code.

We note this here for two reasons. First is transparency. Second, we want to highlight how LLM usage enhanced and expanded the work. The first version of this work focused solely on the GMM approach with simpler examples. The use of an LLM enabled us to significantly expand the scope to include flow matching, and more sophisticated examples and visualization. The LLM also offers a new opportunity to engage with the work by "chatting" with the repository. We provide a `SKILL.md` file in the supporting repository that provides "Expert guidance for solving optimization problems using generative models (GMM and Flow Matching). Use when users need to solve optimization, inverse problems, or find feasible solutions under constraints using probabilistic sampling approaches.".

Results and discussion

Basic concepts of generative modeling

In generative modeling we have two options. First, we can develop a model of the joint probability distribution between inputs x and outputs y. Then we can conditionally generate samples, e.g. at fixed x or fixed y by updating the joint probability to reflect the residual degrees of freedom. The Gaussian Mixture Model is a generative model like this where there are analytical formulas to update the probabilities. We illustrate this conceptually below in Figure 2.

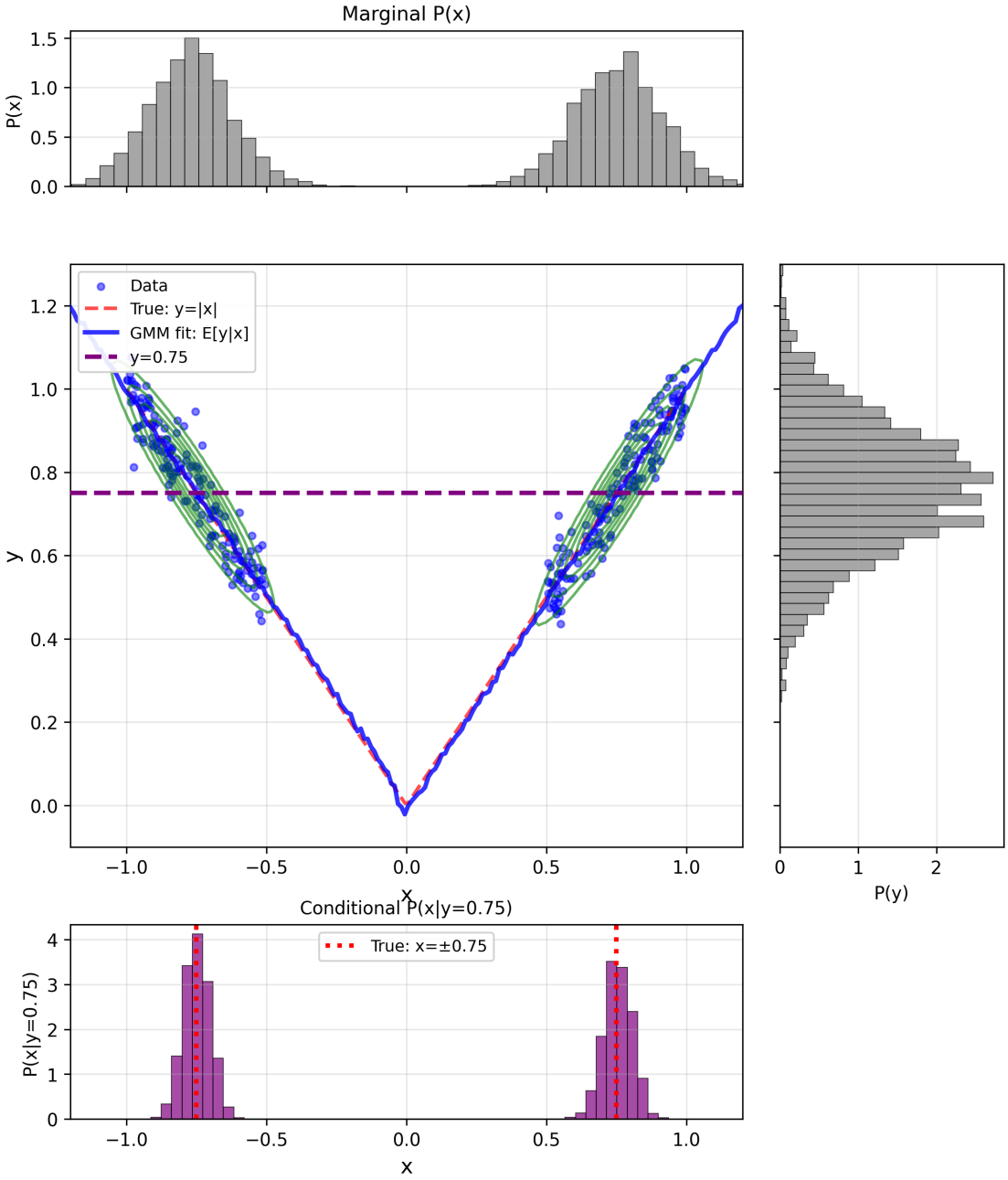


Figure 2: GMM fit for $y = |x|$ with missing data in the region $-0.5 \leq x \leq 0.5$. The central panel shows the data (blue points), true function (red dashed), GMM fit $E[y|x]$ (blue solid line), and GMM density contours (green). The marginal distributions $P(x)$ (top) and $P(y)$ (right) show the overall data distribution. The bottom panel shows the conditional distribution $P(x|y = 0.75)$, which is bimodal with modes at $x \approx \pm 0.75$, demonstrating GMM's ability to capture multi-modal inverse solutions despite missing data.

The generative approach in conditional flow matching is different. Instead of updating the joint probability distributions these models learn a conditional velocity field that transforms one distribution (often a simple normal distribution) into the target distribution. We show a result for this below in Figure 3 for the same problem we solved above with the GMM.

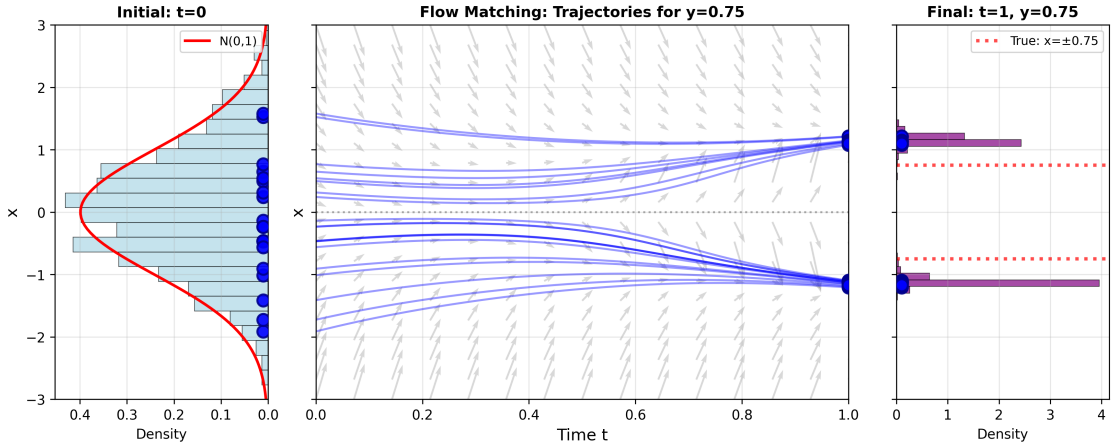


Figure 3: Flow Matching trajectories for $y = |x|$ with missing data. Left panel shows the initial noise distribution $N(0, 1)$ at $t = 0$. Center panel shows 20 trajectories evolving from noise to data, conditioned on $y = 0.75$, with gray arrows indicating the learned velocity field $v(x, t, y = 0.75)$. Right panel shows the final conditional distribution $P(x|y = 0.75)$ at $t = 1$, which is bimodal with modes at $x \approx \pm 0.75$, matching the true inverse solutions. Flow Matching learns to transform a simple Gaussian into a complex bimodal distribution conditioned on the target value.

Root finding

In root finding we seek values of x where $f(x) = 0$. Traditional approaches include bisection, Newton-Raphson iteration, and polynomial root solvers. Here we demonstrate a generative approach where we learn the joint distribution $p(x, f(x))$ and then condition on $f(x) = 0$ to generate samples of the roots.

A practically relevant example is finding compressibility factors from the Peng-Robinson equation of state (PR-EOS), widely used in chemical engineering for vapor-liquid equilibrium calculations. In compressibility factor form, the PR-EOS is a cubic equation:

$$Z^3 - (1 - B)Z^2 + (A - 3B^2 - 2B)Z - (AB - B^2 - B^3) = 0 \tag{15}$$

where $A = aP/(R^2T^2)$ and $B = bP/(RT)$ are dimensionless parameters computed from the substance's critical properties. In the two-phase region, this cubic has three real roots: the liquid compressibility factor Z_L, an unstable intermediate root, and the vapor compressibility factor Z_V. For propane at $T = 300$ K and $P = 10$ bar (in the two-phase region), we generate training data by evaluating the cubic residual $r(Z) = Z^3 - (1 - B)Z^2 + (A - 3B^2 - 2B)Z - (AB - B^2 - B^3)$ over the domain $Z \in [0.01, 0.99]$. We then fit generative models to the joint data $[Z, r(Z)]$.

To find the roots, we condition on $r(Z) = 0$ and sample from the resulting conditional distribution $p(Z \mid r = 0)$. Figure 4 shows the conditional probability density, which exhibits three distinct peaks corresponding to the liquid ($Z_L \approx 0.046$), unstable ($Z \approx 0.20$), and vapor ($Z_V \approx 0.82$) roots. Cluster analysis of the samples yields:

Root	GMM estimate	Analytical
Z_{liquid}	0.0459 ± 0.0006	0.0459
Z_{unstable}	0.1962 ± 0.0095	0.1966
Z_{vapor}	0.8201 ± 0.0044	0.8208

The generative approach naturally discovers all three roots simultaneously, whereas iterative methods like Newton-Raphson find only the root nearest to the initial guess and require multiple starting points to find all roots. The generative framework extends naturally to finding complex roots by treating real and imaginary parts as separate dimensions. An example of this can be found in the supporting information.

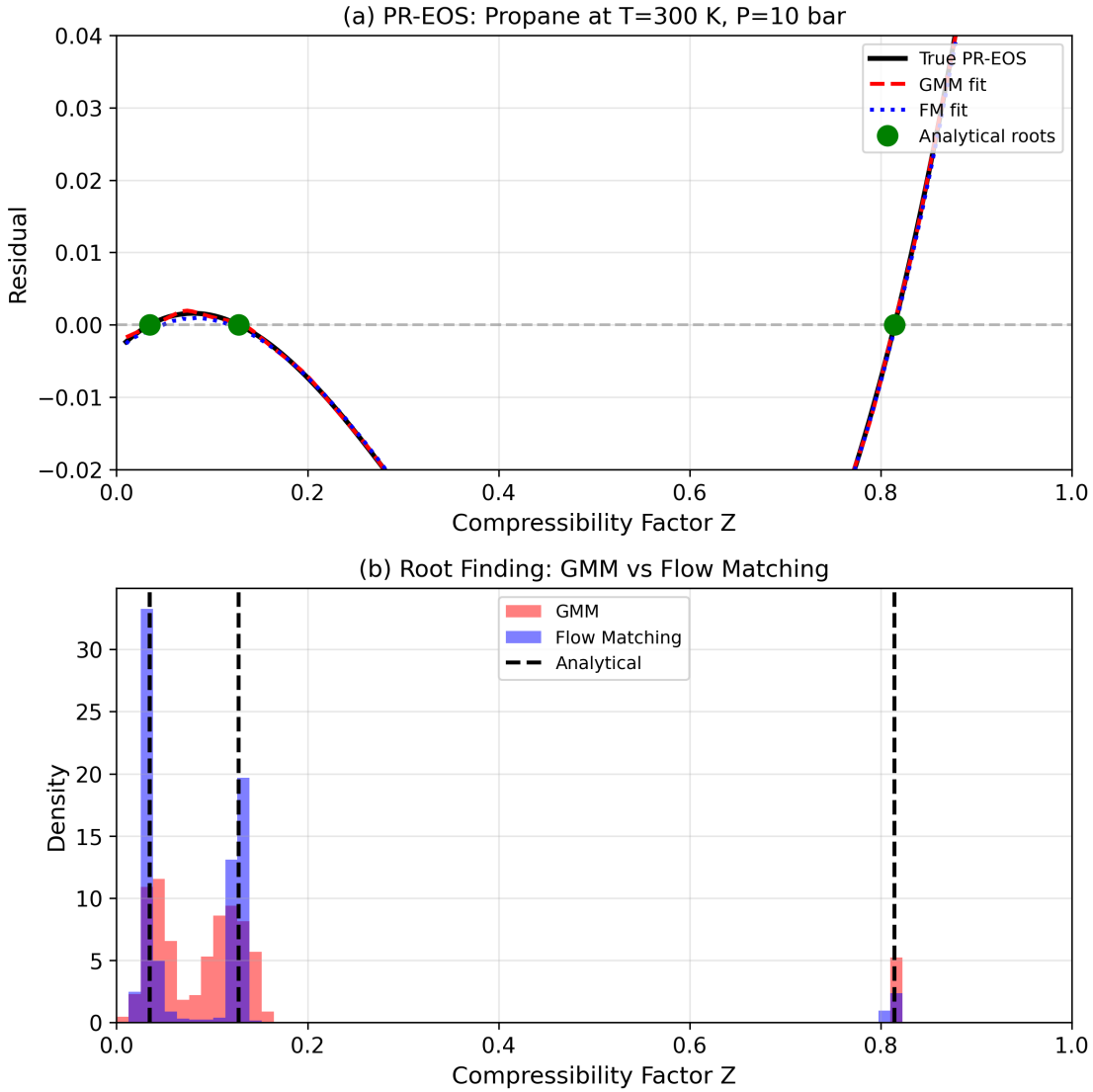


Figure 4: Comparison of Gaussian Mixture Model (GMM) and Flow Matching (FM) approaches for root finding in the Peng-Robinson equation of state. (a) The PR-EOS cubic residual for propane at T = 300 K and P = 10 bar, showing the true function (black solid line) along with the learned fits from GMM (red dashed) and FM (blue dotted). Both generative models accurately capture the functional relationship between compressibility factor Z and the cubic residual. Green circles indicate the three analytical roots corresponding to liquid ($Z_L = 0.049$), unstable ($Z_{unstable} = 0.161$), and vapor ($Z_V = 0.803$) phases. (b) Distribution of roots obtained by conditioning each model on residual = 0. Both methods successfully identify all three roots, with the GMM producing slightly sharper distributions due to its analytical conditioning, while FM generates samples through numerical integration of the learned velocity field.

Optimization with generative models

Consider consecutive first-order reactions in a batch reactor:

$$A \xrightarrow{k_1} B \xrightarrow{k_2} C \tag{16}$$

where B is the desired product and C is an unwanted byproduct. The following parameters were used: $k_1 = 0.5$ min^{-1} (A $\rightarrow$ B), $k_2 = 0.2$ min^{-1} (B $\rightarrow$ C), $C_{A0} = 1.0$ mol/L. This leads to an optimal solution of $\tau^* \approx 3.05$ min and $C_B(\tau^*) \approx 0.549$ mol/L.

The concentration of intermediate B at time τ is given by:

$$C_B(\tau) = \frac{C_{A0} k_1}{k_2 - k_1} \left(e^{-k_1 \tau} - e^{-k_2 \tau} \right) \tag{17}$$

where C_{A0} is the initial concentration of A, and k_1, k_2 are the rate constants.

The optimization problem is to find the residence time τ^* that maximizes the concentration of intermediate B:

$$\max_{\tau} C_B(\tau) \tag{18}$$

The first-order necessary condition for a maximum is:

$$\frac{dC_B}{d\tau} = C_{A0} k_1 \frac{k_2 e^{-k_2 \tau} - k_1 e^{-k_1 \tau}}{k_2 - k_1} = 0 \tag{19}$$

Solving equation (19) yields the analytical optimum:

$$\tau^* = \frac{\ln(k_1/k_2)}{k_1 - k_2} \tag{20}$$

The second derivative confirms this is a maximum:

$$\left. \frac{d^2 C_B}{d\tau^2} \right|_{\tau = \tau^*} < 0 \tag{21}$$

Rather than solving the optimality condition analytically, the generative approach starts by generating training data consisting of tuples $\left(\tau, C_B(\tau), \frac{dC_B}{d\tau}\right)$, learning the joint distribution $p\left(\tau, C_B, \frac{dC_B}{d\tau}\right)$, then identifying the optimum by sampling from the conditional distribution $p\left(\tau \mid \frac{dC_B}{d\tau} = 0\right)$. This probabilistic inference approach generalizes to complex reaction networks where analytical solutions do not exist.

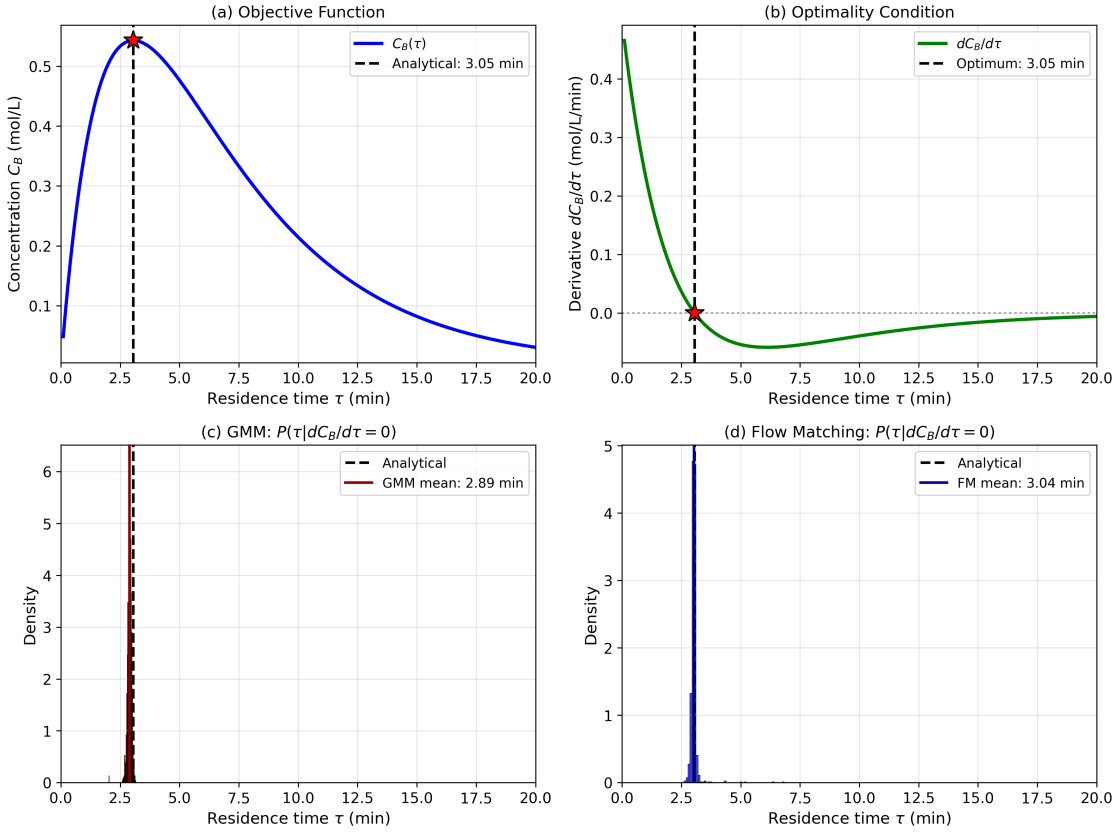


Figure 5: Optimization via conditional sampling for consecutive reactions A $\rightarrow$ B $\rightarrow$ C. (a) Intermediate product concentration $C_B(\tau)$ shows maximum at $\tau^* \approx 3.05$ min. (b) First derivative $dC_B/d\tau$ crosses zero at the optimum. (c-d) GMM and flow matching identify the optimum by sampling from $P(\tau|dC_B/d\tau = 0)$, producing narrow distributions (red/blue solid lines show means) that closely match the analytical solution (black dashed line). Both methods successfully solve the optimization problem through probabilistic inference, demonstrating that conditioning on the optimality condition (derivative = 0) yields the optimal residence time with inherent uncertainty quantification.

Optimization with equality constraints

With equality constraints it may not be possible to solve optimization problems by conditioning on the objective function's derivative being zero; if a constraint is active, the derivative of the objective function may not equal zero at the solution. To solve this generatively, we turn to the Lagrangian function that transforms the constrained problem to an unconstrained problem where the derivatives of the Lagrangian are equal to zero. We construct this function, generate samples of the inputs (including the Lagrange multiplier λ values) and the derivatives of the Lagrangian. Then we find solutions by conditioning on $\nabla\mathcal{L} = 0$.

We illustrate this with a practical chemical engineering problem: finding a gasoline blend closest to a target recipe while meeting an octane specification. A refinery blends three components (reformate, isomerate, and alkylate) and wants to minimize deviation from a preferred blend composition $(x_1^*, x_2^*, x_3^*) = (0.4, 0.4, 0.2)$ while satisfying quality constraints. The problem is formulated as:

$$\begin{aligned} \min_{x_1,x_2,x_3} \quad & (x_1 - 0.4)^2 + (x_2 - 0.4)^2 + (x_3 - 0.2)^2 \\ \text{subject to} \quad & \text{RON}_1 x_1 + \text{RON}_2 x_2 + \text{RON}_3 x_3 = \text{RON}_{\text{target}} \\ & x_1 + x_2 + x_3 = 1 \end{aligned} \tag{22}$$

where x_i are the volume fractions of each component and RON_i are the Research Octane Numbers. We use representative values: reformate ($\text{RON}_1 = 95$), isomerate ($\text{RON}_2 = 82$), and alkylate ($\text{RON}_3 = 94$), with a target octane of $\text{RON}_{\text{target}} = 87$ (regular gasoline specification). The quadratic objective ensures an interior solution exists where all blend fractions are positive, which is essential for the Lagrangian approach to work correctly.

The Lagrangian for this problem with two equality constraints is:

$$\mathcal{L}(x_1, x_2, x_3, \lambda_1, \lambda_2) = \sum_{i=1}^{3}(x_i - x_i^*)^2 + \lambda_1(g_1) + \lambda_2(g_2) \tag{23}$$

where $g_1 = \text{RON}_1 x_1 + \text{RON}_2 x_2 + \text{RON}_3 x_3 - \text{RON}_{\text{target}}$ is the octane constraint and $g_2 =$

$x_1 + x_2 + x_3 - 1$ is the mass balance constraint.

The KKT conditions require all five partial derivatives of the Lagrangian to equal zero:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial x_1} &= 2(x_1 - 0.4) + 95\lambda_1 + \lambda_2 = 0 \\
\frac{\partial \mathcal{L}}{\partial x_2} &= 2(x_2 - 0.4) + 82\lambda_1 + \lambda_2 = 0 \\
\frac{\partial \mathcal{L}}{\partial x_3} &= 2(x_3 - 0.2) + 94\lambda_1 + \lambda_2 = 0 \\
\frac{\partial \mathcal{L}}{\partial \lambda_1} &= g_1 = 0 \\
\frac{\partial \mathcal{L}}{\partial \lambda_2} &= g_2 = 0
\end{aligned} \tag{24}$$

The generative approach transforms this constrained optimization into a sampling problem. We generate samples of $(x_1, x_2, x_3, \lambda_1, \lambda_2)$ along with their corresponding Lagrangian derivatives (computed using automatic differentiation), train a GMM on this joint distribution, and then condition on all five derivatives being zero to recover the optimal blend.

The reference solution from `scipy.optimize.minimize` with the SLSQP method yields $x_1 = 0.284$, $x_2 = 0.607$, $x_3 = 0.109$. Conditioning the GMM on $\nabla\mathcal{L} = 0$ recovers this solution with high accuracy: GMM estimates of $x_1 = 0.284 \pm 0.004$, $x_2 = 0.607 \pm 0.008$, $x_3 = 0.109 \pm 0.003$. The optimal blend deviates from the target to satisfy the octane constraint, using more isomerate (which has the lowest octane) and less reformate than the target recipe.

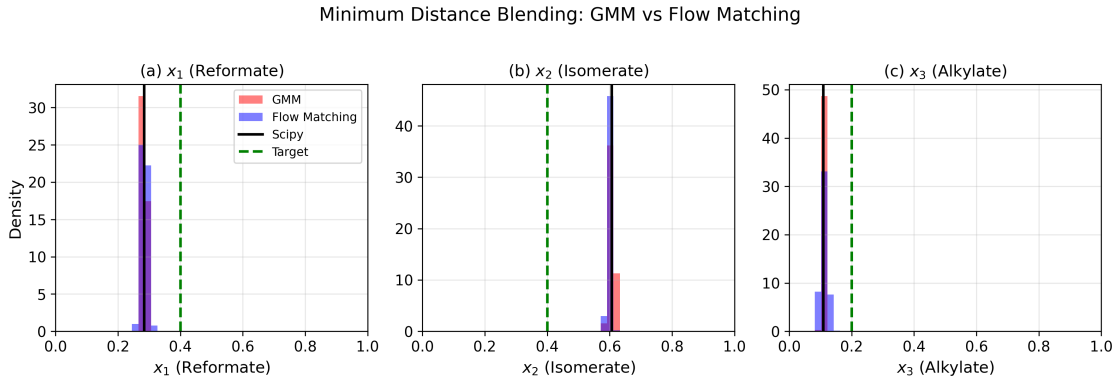


Figure 6: Comparison of GMM and Flow Matching for equality-constrained optimization of a gasoline blending problem. The objective is to find the blend composition closest to a target recipe ($x_1^* = 0.4$, $x_2^* = 0.4$, $x_3^* = 0.2$) while satisfying an octane specification (RON = 87) and mass balance constraint. Panels (a)-(c) show the sample distributions for each blend fraction obtained by conditioning on the KKT conditions ($\nabla\mathcal{L} = 0$). Red histograms: GMM samples; blue histograms: Flow Matching samples; solid black lines: scipy SLSQP reference solution; dashed green lines: target blend values. Both generative methods produce distributions centered near the optimal solution, with GMM showing slightly tighter distributions than Flow Matching. The optimal blend deviates from the target to satisfy the octane constraint, using more isomerate (x_2) and less reformate (x_1) than targeted.

Constrained optimization with an inequality

Inequality constraints bring a new challenge to using a generative method; there could be a range of possible values that could satisfy the solution (that is the nature of inequality), so it is not possible to condition on one set of values like it is with the equality constraint. A benefit of the conventional `minimize` function in `scipy` is one simply changes the constraint type.

We cannot use the Lagrange method for an inequality constraint the same way we did for an equality constraint. Instead we consider the barrier method.[40] Recalling the barrier function for the inequality-constrained optimization problem defined as in Eq. 25, in which the objective function incorporates the set of inequality constraints $\mathcal{I}$ with the form illustrated below.

$$f(x) - \mu \sum_{i \in \mathcal{I}} \log c_i(x) \tag{25}$$

We know from classic optimization that when $\mu \to 0$ the barrier terms vanishes and the adapted problem in Eq. 25 collapses into the original inequality problem. Now, the main task is to specify a μ value here and that is usually selected iteratively. We do not do that here, but use a value known to be adequate for our proposed approach. It should be possible to iteratively try different values of μ to convergence, but we rather focus on demonstrating that is possible to use a generative approach to inequality constrained optimization problem, as a rich body of literature in inequality constrained optimization already discusses challenges in this area.[5,40]

A physically motivated inequality constraint example arises in chemical reaction equilibrium. Consider the reversible reaction $A + B \rightleftharpoons 2C$ in a batch reactor. Given initial concentrations $C_{A0} = C_{B0} = 1$ mol/L and $C_{C0} = 0$, the concentrations evolve with extent of reaction ξ as:

$$C_A = 1 - \xi, \quad C_B = 1 - \xi, \quad C_C = 2\xi \tag{26}$$

The equilibrium condition requires:

$$K_{eq} = \frac{C_C^2}{C_A \cdot C_B} = \frac{(2\xi)^2}{(1-\xi)^2} \tag{27}$$

Rearranging gives the equilibrium residual that must equal zero:

$$f(\xi) = K_{eq}(1-\xi)^2 - 4\xi^2 = 0 \tag{28}$$

For $K_{eq} = 4$, the analytical solution is $\xi^* = 0.5$, giving equilibrium concentrations $C_A = C_B = 0.5$ mol/L and $C_C = 1.0$ mol/L. The physical constraint that concentrations must be non-negative requires $0 < \xi < 1$.

To solve this constrained root finding problem with a generative approach, we sample ξ values only within the feasible region $(0, 1)$ and compute the equilibrium residual $f(\xi)$ for each sample. We train generative models on the joint distribution $[\xi, f(\xi)]$ and condition on $f(\xi) = 0$ to find the equilibrium extent. The constraint is naturally enforced by the sampling domain. Both GMM and Flow Matching successfully identify $\xi \approx 0.5$, as shown in Figure 7.

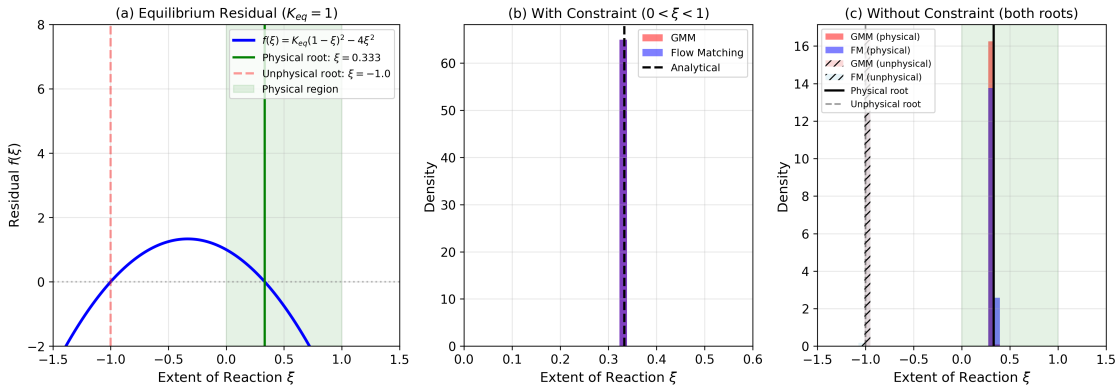


Figure 7: Reaction equilibrium $A + B \rightleftharpoons 2C$ with physical constraints requiring non-negative concentrations. (a) The equilibrium residual $f(\xi) = K_{eq}(1-\xi)^2 - 4\xi^2$, which equals zero at the equilibrium extent $\xi^* = 0.5$. (b) Distributions of ξ values obtained by conditioning GMM (red) and Flow Matching (blue) on $f(\xi) = 0$. Both methods successfully identify the analytical solution (black dashed line).

This example demonstrates that constrained root finding problems integrate naturally with the generative approach. By sampling only within the feasible region and conditioning on the function value being zero, we find physically meaningful solutions. This approach generalizes to more complex reaction networks where analytical solutions may not exist.

Parameter estimation with generative models

Parameter estimation in regression is a kind of inverse problem. Usually we consider this in the context of optimization as finding a set of parameters (inputs) that minimize an objective function (outputs) of the errors between a model and observations. To recast this idea for a generative model, we consider the dataset as having columns of parameters (inputs) and observations (outputs). The observations could be derivatives of the objective function,

and then one conditions on those being zero like we did before. We take an alternative approach here, and consider the observations to be some output of a model. Then, given an observation, we condition on that to obtain the parameters most consistent with that observation. This approach is fundamentally different than regression where parameters are estimated by minimizing some function of pooled errors.

For a concrete example, we use a series reaction: A $\rightarrow$ B $\rightarrow$ C in a plug flow reactor. The observations we have will be the exit molar flow rates of A and B. We use a inlet molar flow, fixed volume and volumetric flow rate. The system is governed by these two differential equations:

$$dF_a/dV = -k_1 F_a/\nu$$

$$dF_b/dV = k_1 F_a/\nu - k_2 F_b/\nu$$

We sample a range of rate constants and compute the exit molar flows of A and B. This data set is what we will use to build a generative model next. We generate samples of k_1, k_2 in the range of 0.1..3. We can see what this data looks like in Figure 8.

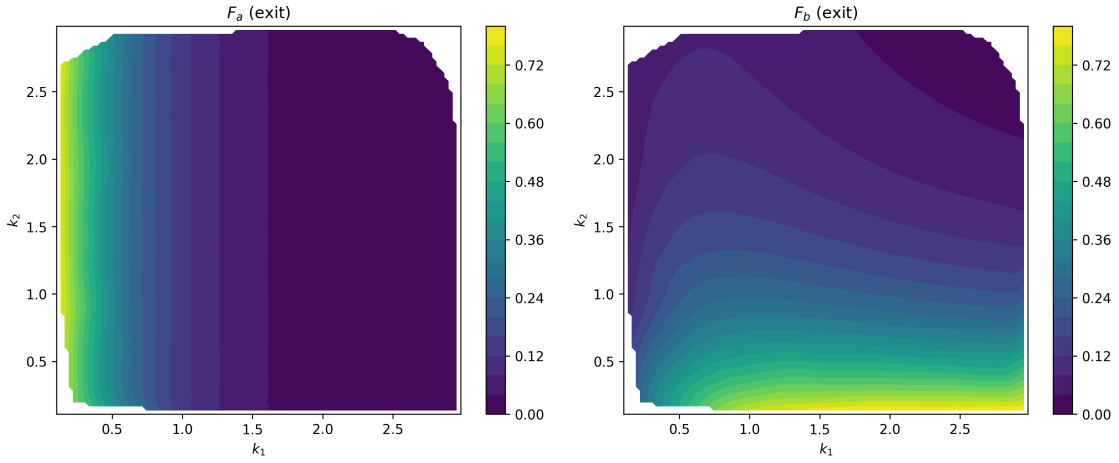


Figure 8: Contour plots of the relationships between k_1 and k_2 and a) F_A and b) F_B.

It is evidently not enough to specify one of these values to uniquely specify the rate constants. In F_A, there are near vertical isolines, and in F_B other isolines. For example, for $k_1 = 2$, F_a will be between 0-0.02 for a broad range of k_2 values, and which value of k_2 that is chosen depends on the value of F_b that is observed. If you see low F_b, k_2 will be closer to

2, and if F_b is high, then k_2 is closer to 0.2.

For generative parameter estimation we can consider the following joint probability distribution $[k_1, k_2, F_A, F_B]$, and then condition the distribution on observed output concentrations to generate the set of parameters that are consistent that those observations. For illustrative purposes we estimate parameters for two scenarios at the exit:

1. A low molar flow of A with low molar flow of B, i.e. both reactions are fast.

2. A low molar flow of A with high molar flow of B, i.e. reaction 1 is fast, and reaction 2 is slow.

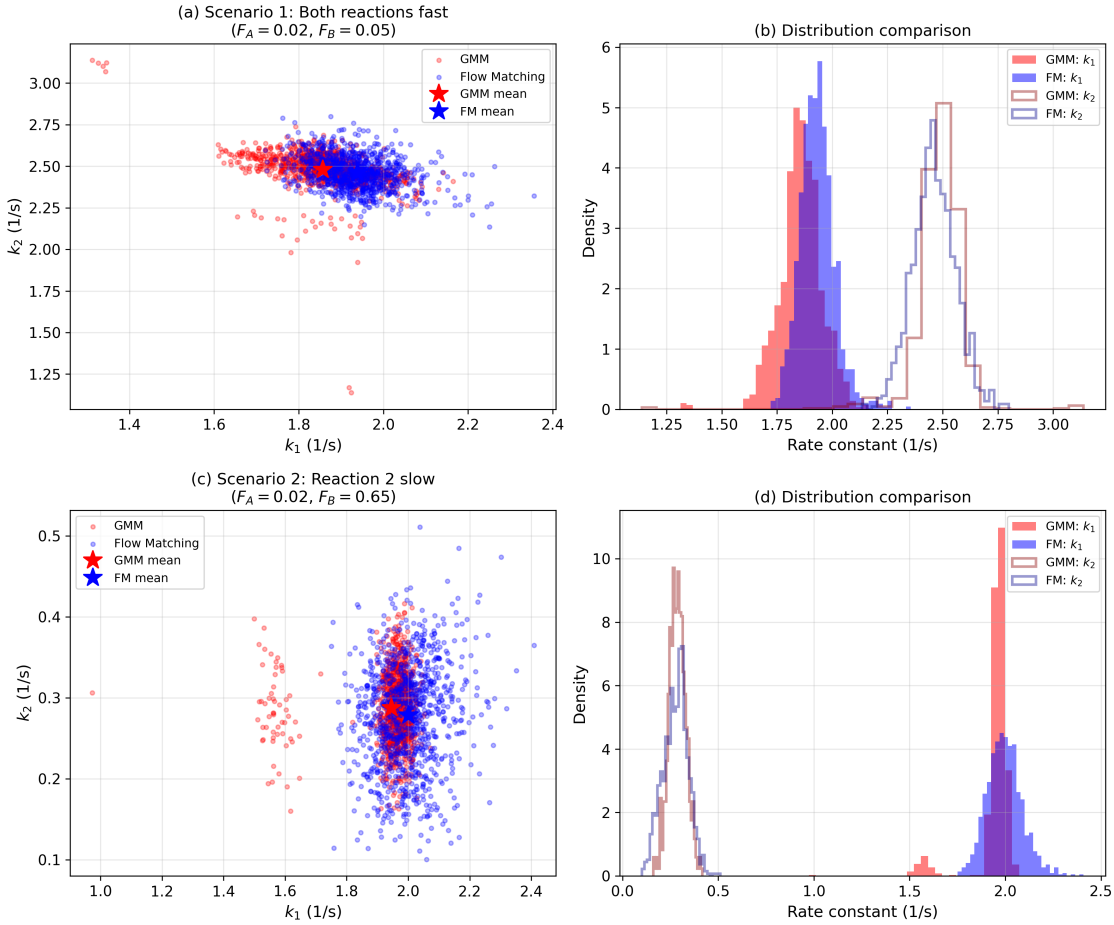


Figure 9: Estimating rate constants (k_1, k_2) from reactor exit flows (F_A, F_B) using GMM and Flow Matching. Left panels show joint distributions in parameter space; right panels show marginal distributions. (a-b) Scenario 1 ($F_A = 0.02$, $F_B = 0.05$): Both reactions fast, requiring high k_1 and k_2. (c-d) Scenario 2 ($F_A = 0.02$, $F_B = 0.65$): First reaction fast, second slow, requiring high k_1 and low k_2. Both methods correctly identify the parameter regimes. GMM shows minor multimodality due to the non-unique inverse problem, while Flow Matching produces smoother distributions. Mean estimates (stars) accurately reproduce the target observations.

It is also possible to estimate an uncertainty of sorts. The distribution of the estimated rate constants is due to the surrogate models not fitting the data perfectly well, as seen in the parity plot. It is also possible to use this approach to map uncertainties in the inputs to the outputs and vice versa. For example, if one has the confidence intervals of a parameter they can map that range onto outputs with this approach. We illustrate this kind of mapping in Sec. , and previously showed an approach using inverse mapping.[41]

Mapping input and output spaces

We previously developed a mapping approach based on differential equations.[42] We illustrate here that this can also be achieved by a generative model. We expand this idea to using a generative model here. The goal is to map a desired output space in the concentration of A, B from a CSTR to the input space of the radius of a CSTR, and the temperature (in units of RT) that achieves it. We have a design with fixed height, and we can modify the radius and temperature (via RT). There are two reactions: A $\rightarrow$ B, that is second order in A, and A $\rightarrow$ C, that is first order in A. We seek a design space with desired amounts of A and B in the exit. We choose a circular region in the output space, and map it to the input space.

Given a radius, we have $V = \pi R^2 H$ and given an RT we have these rate constants for the two forward reactions: $k_1 = \exp(-3/RT)$, $k_2 = \exp(-10/RT)$.

We start with a mole balance:

$0 = F_{A0} - F_A + r_A V$

Some rearrangement leads to:

$0 = V k_1 C_A^2 + (v_0 + V k_2) C_A - F_{A0}$

which we can solve with the quadratic equation to get C_A as a function of (R, RT), and then from a mole balance on B, we get

$C_B = k_1 C_A^2 V / v_0$

Thus, we can analytically map the input space to the output space as shown in Figure 10. We do this by brute force sampling here and show a how a circular region in the output space maps to the input space. We choose this example because it is easy to do this analytically, but this forward mapping can also be done in nonlinear programs.

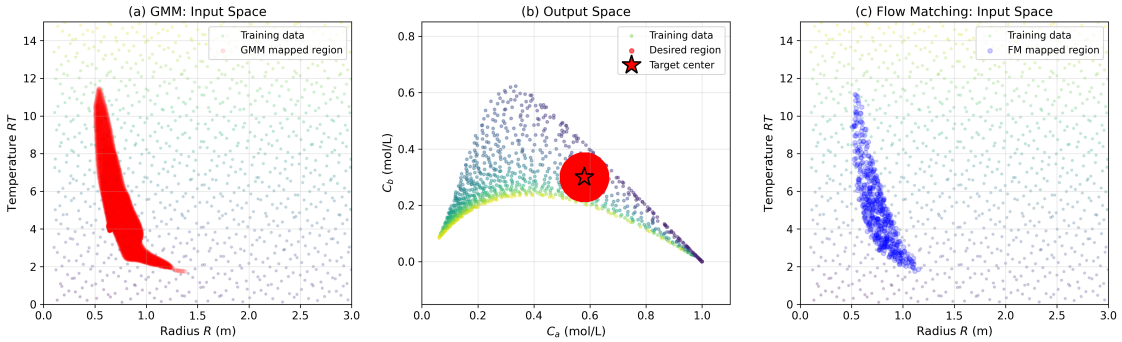


Figure 10: Mapping a desired concentration region (b, red circle: Ca = 0.58, Cb = 0.30 ± 0.08 mol/L) to feasible design parameters using (a) GMM and (c) Flow Matching. Both methods identify the irregular, non-convex region in (R, RT) space that produces the target concentrations. Background points show training data colored by distance from origin. The complex shape of the mapped region reveals the nonlinear relationship between reactor size, temperature, and exit concentrations, demonstrating that multiple design configurations can achieve the same production target. This inverse mapping capability is unique to generative models.

This is not the same as learning a forward or backward model, e.g. a neural network or Gaussian process model. This is an approximate, data-driven mapping through a joint distribution model. It is not as smooth as the mapping by ODEs, or an analytical mapping, but it might be suitable for some applications. Building the model from more samples would improve the mapping accuracy. It is also possible to map in the forward direction, e.g. from a desired input space to an output space. This has application in uncertainty propagation.

There is a connection between the mapping of input and output spaces by differential equations we previously developed[42] and the generative models we use here. We used a differentiable model to derive a set of ODEs that connect the inputs and outputs. We considered a point-wise mapping where one could use the ODEs to transform a single point in one space to the other space. In flow-mapping one uses machine learning to learn the vector field that defines the ODEs that transform the input distribution to the output distribution. Then, by sampling from one space it is possible to transform that sample into the other space. In this way, we might view these models as data-driven versions of the model-driven mapping idea we developed.

Conclusions

We have illustrated several different examples of using generative models in optimization tasks that ranged from root finding, optimization, optimization with constraints, parameter estimation and state mapping. In each case the solution is obtained by conditional generation from a distribution trained from samples of inputs and outputs. In root-finding, the outputs are function values. In optimization, the outputs include derivatives. For constrained optimization, the outputs were derivatives of an augmented Lagrange function. In parameter estimation, we used observations for the outputs. The identification of the input / output pairs that allow one to generate conditional samples from specific values is the key to using generative models this way.

There are a few ways this generative approach could fail. First, if it is hard to model the distribution, or learn the transformation the approach would obviously fail. Second, even if it succeeds, there is no guarantee one finds all the possible solutions, even of the surrogate function. The surrogate function may not have sufficient accuracy to find the true solutions of the real system, or may have artifacts that lead to false solutions. The models are built from and the solutions are found by sampling. For well-behaved broad distributions, sampling should work very well. For poorly-behaved distributions, e.g. where there is a very sharp probability, it is possible to miss a result, both in building the model and sampling it for solutions.

It is evident this is not a panacea solution to optimization. Conventional methods may be more efficient and accurate for some problems, especially when there is a model available. Even regular machine learned models with conventional methods may be better when they have desirable properties, e.g. converting a Relu neural network to a mixed integer program. It remains unknown if we can find a way to include inequality constraints. The whole approach resembles using a surrogate model, and is limited in the same ways; the solutions are to the surrogate model, and not the underlying true model. That is a feature when there is no actual model, but rather data. Sampling strategies can be essential to success, and

there are not yet established methods to efficiently sample the data one needs. Since this is ultimately a data-driven approach, it is not likely to be accurate in sample generation with conditions that are out of distribution.

Generative models currently do not have the same tools for analysis that regular models have. It is not obvious, for example, how one might define the derivatives of a generative model. In principle, the derivative would itself be a distribution. As with other machine learning models, data-driven generative models should only be expected to be valid for "in-domain" sampling, but in high-dimensional spaces it is not always obvious how to quantify what is in or out of domain. Although one can use sampling to quantify variance in predictions, that variance is a combination of model inadequacy and noise in the data.

Nevertheless, this work shows that generative methods have enormous possibility in a broad range of applications we have not previously considered them for in optimization, as surrogate models, and likely others we have not thought of yet. There are many generative models in existence today, and new ones are still being found. It seems likely these will mitigate problems we have illustrated in this work, and make new approaches possible in the future. We are most excited by the new way of thinking about problems that this approach now enables.

Acknowledgments

JRK acknowledges Carl Laird for suggesting the barrier function approach for problems with inequality constraints.

References

(1) Dantzig, G. B. *A history of scientific computing*; 1990; pp 141–151.

(2) Han, S. P. A globally convergent method for nonlinear programming. *Journal of Optimization Theory and Applications* **1977**, *22*, 297–309.

(3) Powell, M. J. D. A fast algorithm for nonlinearly constrained optimization calculations. Numerical Analysis. Berlin, Heidelberg, 1978; pp 144–157.

(4) Wilson, R. B. A Simplicial Algorithm for Concave Programming. Doctor of Business Administration dissertation, Harvard Business School, Cambridge, MA, 1963; Discipline: Business Administration. Advisor: Colin Lukens.

(5) Wächter, A.; Biegler, L. T. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical programming* **2006**, *106*, 25–57.

(6) Bonami, P.; Biegler, L. T.; Conn, A. R.; Cornuéjols, G.; Grossmann, I. E.; Laird, C. D.; Lee, J.; Lodi, A.; Margot, F.; Sawaya, N.; Wächter, A. An algorithmic framework for convex mixed integer nonlinear programs. *Discrete Optimization* **2008**, *5*, 186–204, In Memory of George B. Dantzig.

(7) Grossmann, I. E.; Ruiz, J. P. Generalized Disjunctive Programming: A Framework for Formulation and Alternative Algorithms for MINLP Optimization. Mixed Integer Nonlinear Programming. New York, NY, 2012; pp 93–115.

(8) Floudas, C.; Akrotirianakis, I.; Caratzoulas, S.; Meyer, C.; Kallrath, J. Global optimization in the 21st century: Advances and challenges. *Computers & Chemical Engineering* **2005**, *29*, 1185–1202, Selected Papers Presented at the 14th European Symposium on Computer Aided Process Engineering.

(9) Floudas, C. A.; Gounaris, C. E. A review of recent advances in global optimization. *Journal of Global Optimization* **2009**, *45*, 3–38.

(10) Boukouvala, F.; Misener, R.; Floudas, C. A. Global optimization advances in Mixed-Integer Nonlinear Programming, MINLP, and Constrained Derivative-Free Optimization, CDFO. *European Journal of Operational Research* **2016**, *252*, 701–727.

(11) Virtanen, P. et al. Scipy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* **2020**, *17*, 261–272.

(12) Bynum, M. L.; Hackebeil, G. A.; Hart, W. E.; Laird, C. D.; Nicholson, B. L.; Siirola, J. D.; Watson, J.-P.; Woodruff, D. L. *Pyomo - Optimization Modeling in Python*; Springer Optimization and Its Applications; Springer International Publishing, 2021; p nil.

(13) Beal, L.; Hill, D.; Martin, R.; Hedengren, J. Gekko Optimization Suite. *Processes* **2018**, *6*, 106.

(14) Gurobi Optimization, LLC Gurobi Optimizer Reference Manual. 2024; https://www.gurobi.com.

(15) Andersen, E. D.; Andersen, K. D. In *High Performance Optimization*; Frenk, H., Roos, K., Terlaky, T., Zhang, S., Eds.; Springer US: Boston, MA, 2000; pp 197–232.

(16) Dunning, I.; Huchette, J.; Lubin, M. Jump: a Modeling Language for Mathematical Optimization. *SIAM Review* **2017**, *59*, 295–320.

(17) Biegler, L. T.; Grossmann, I. E. Retrospective on optimization. *Computers & Chemical Engineering* **2004**, *28*, 1169–1192.

(18) NEOS Guide Optimization Problem Types. 2025; https://neos-guide.org/guide/types/, Accessed: 2025-04-22.

(19) GM, H.; Gourisaria, M. K.; Pandey, M.; Rautaray, S. S. A Comprehensive Survey and Analysis of Generative Models in Machine Learning. *Computer Science Review* **2020**, *38*, 100285.

(20) Ardizzone, L.; Kruse, J.; Rother, C.; Köthe, U. Analyzing Inverse Problems with Invertible Neural Networks. International Conference on Learning Representations. 2019.

(21) Dasgupta, A.; Ramaswamy, H.; Murgoitio-Esandi, J.; Foo, K. Y.; Li, R.; Zhou, Q.; Kennedy, B. F.; Oberai, A. A. Conditional Score-Based Diffusion Models for Solving Inverse Elasticity Problems. *Computer Methods in Applied Mechanics and Engineering* **2025**, *433*, 117425.

(22) Kitchin, J. Solving an inverse problem with generative models. 2025; https://doi.org/10.26434/chemrxiv-2025-nl9gl.

(23) Alcazar, J.; Vakili, M. G.; Kalayci, C. B.; Perdomo-Ortiz, A. Enhancing Combinatorial Optimization With Classical and Quantum Generative Models. *Nature Communications* **2024**, *15*, 2761.

(24) Yao, M. S.; Zeng, Y.; Bastani, H.; Gardner, J.; Gee, J. C.; Bastani, O. Generative Adversarial Model-Based Optimization Via Source Critic Regularization. *arXiv* **2024**, 2402.06532.

(25) Becker, J.; Wahle, J. P.; Gipp, B.; Ruas, T. Text Generation: a Systematic Literature Review of Tasks, Evaluation, and Challenges. *arXiv* **2024**, 2405.15604.

(26) Elasri, M.; Elharrouss, O.; Al-Maadeed, S.; Tairi, H. Image Generation: a Review. *Neural Processing Letters* **2022**, *54*, 4609–4646.

(27) Yang, L.; Cheung, N.-M.; Li, J.; Fang, J. Deep Clustering by Gaussian Mixture Variational Autoencoders With Graph Embedding. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). 2019.

(28) Viroli, C.; McLachlan, G. J. Deep Gaussian mixture models. *Statistics and Computing* **2019**, *29*, 43–51.

(29) Bishop, C. M.; Nasrabadi, N. M. *Pattern recognition and machine learning*; Springer, 2006; Vol. 4.

(30) Fabisch, A. gmr: Gaussian Mixture Regression. *Journal of Open Source Software* **2021**, *6*, 3054.

(31) Deisenroth, M. P.; Faisal, A. A.; Ong, C. S. *Mathematics for machine learning*; Cambridge University Press, 2020.

(32) Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; Vanderplas, J.; Passos, A.; Cournapeau, D.; Brucher, M.; Perrot, M.; Duchesnay, E. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* **2011**, *12*, 2825–2830.

(33) Ghahramani, Z.; Jordan, M. Supervised learning from incomplete data via an EM approach. Advances in Neural Information Processing Systems. 1993.

(34) Lipman, Y.; Chen, R. T. Q.; Ben-Hamu, H.; Nickel, M.; Le, M. Flow Matching for Generative Modeling. *International Conference on Learning Representations (ICLR)* **2023**,

(35) Tong, A.; Malkin, N.; Huguet, G.; Zhang, Y.; Rector-Brooks, J.; Fatras, K.; Wolf, G.; Bengio, Y. Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport. *Transactions on Machine Learning Research* **2023**,

(36) Chen, R. T. Q.; Rubanova, Y.; Bettencourt, J.; Duvenaud, D. K. Neural Ordinary Differential Equations. *Advances in Neural Information Processing Systems (NeurIPS)* **2018**, *31*.

(37) Kingma, D. P.; Ba, J. Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations (ICLR)* **2015**,

(38) Song, Y.; Sohl-Dickstein, J.; Kingma, D. P.; Kumar, A.; Ermon, S.; Poole, B. Score-Based Generative Modeling through Stochastic Differential Equations. *International Conference on Learning Representations (ICLR)* **2021**,

(39) Papamakarios, G.; Nalisnick, E.; Rezende, D. J.; Mohamed, S.; Lakshminarayanan, B. Normalizing Flows for Probabilistic Modeling and Inference. *Journal of Machine Learning Research* **2021**, *22*, 1–64.

(40) Nocedal, J.; Wright, S. *Numerical Optimization*; Springer Series in Operations Research and Financial Engineering; Springer New York, 2006.

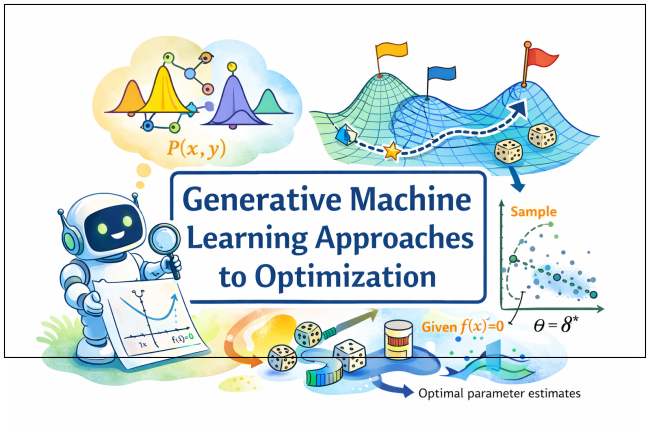
(41) Alves, V.; Laird, C. D.; Lima, F. V.; Kitchin, J. R. Mapping Uncertainty Using Differentiable Programming. *AIChE Journal* **2025**, e18940.

(42) Alves, V.; Kitchin, J. R.; Lima, F. V. An inverse mapping approach for process systems engineering using automatic differentiation and the implicit function theorem. *AIChE Journal* **2023**, e18119.

Data Availability Statement

As Supporting Information we provide https://github.com/KitchinHUB/generative-optimization which contains extensive examples in the form of Jupyter notebooks for the GMM and flow-matching examples, and the notebooks that were used to generate the figures in this manuscript. In addition to the examples discussed in the manuscript there are additional examples on advanced optimization topics and visualization, the SKILL.md file for use with LLMs and automation tools for running the notebooks.

TOC Graphic


$P(x, y)$
Generative Machine Learning Approaches to Optimization
Sample
$\theta = \theta^*$
Given $f(x)=0$
Optimal parameter estimates